



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



人工智能实践教学中心
Artificial Intelligence Practice Teaching Center

AI Agent MCP & SKILLS

学生创新中心 梁晓妮





大模型的发展历程



人工智能实践教学中心
Artificial Intelligence Practice Teaching Center

技术积累阶段

大规模验证时期

迈向Agent 时代

综合Agent 体



2015~2020年

2021~2024年

2024~2025

2026

谷歌、微软、百度



数据挖掘 (图谱分析、
few shots)

百模大战



Coze平台、Cursor、微软
365 copilot、Manus等



“千问助手”打通阿里生
态，一句话就送到
Openclaw、opencode等
开源Agent

A: Transformer架构演进
B: 大模型语料积累
C: CUDA生态及Nvlink技术

GPT-3: Decoder only架构
Chat-GPT: RHLF指令微调

大模型智能提升速度开始收敛
由高智低能->高智高能
产生商业价值

垂直Agent可能成为
阶段性产品
综合Agent, ALL in
one

opencode



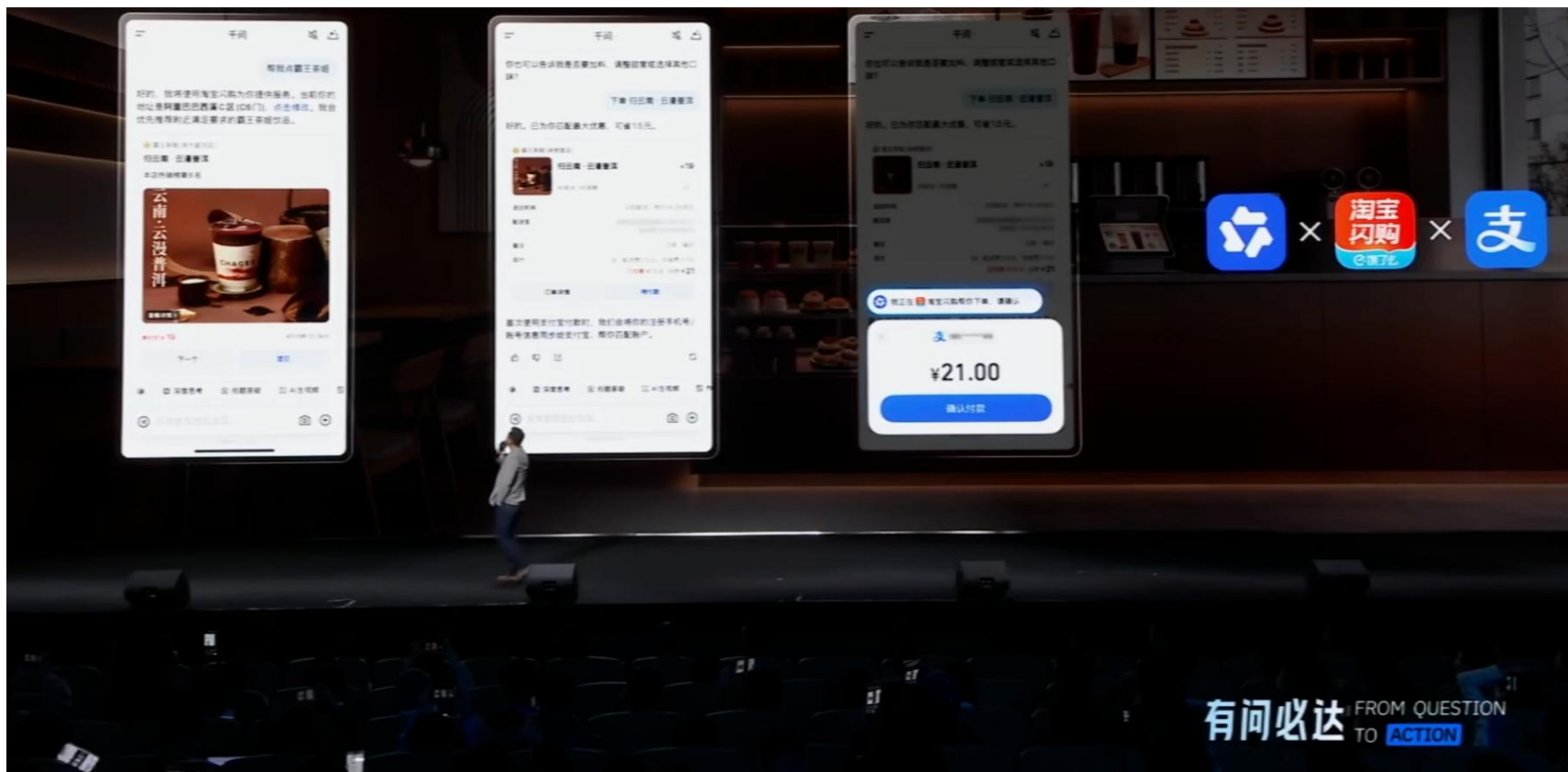


Agent的应用场景



人工智能实践教学中心
Artificial Intelligence Practice Teaching Center

2026年1月15日，千问app发布，有问必达。发票整理、团建行程规划、超级家教、购买推荐、程序编写、写报告等。





Prompt: 设计输入给模型的文本，引导模型产生符合预期的输出，并不修改模型本身的参数。

(零样本提示、少样本提示、思维链、角色扮演)

RAG: 从外部知识库 (如文档、数据库、网页) 中检索相关信息，然后将检索到的内容与用户问题一起提供给模型，让模型基于检索结果生成答案。

Fine-tuning: 在预训练模型的基础上，使用特定领域的数据集对模型进行继续训练，调整模型的部分或全部参数，使其更好地适应目标任务 (全量、高效、指令微调等)。

Tools: 赋予模型调用外部工具的能力，让模型突破语言模型只能生成文本的限制，执行实时计算、访问API、操作软件等。

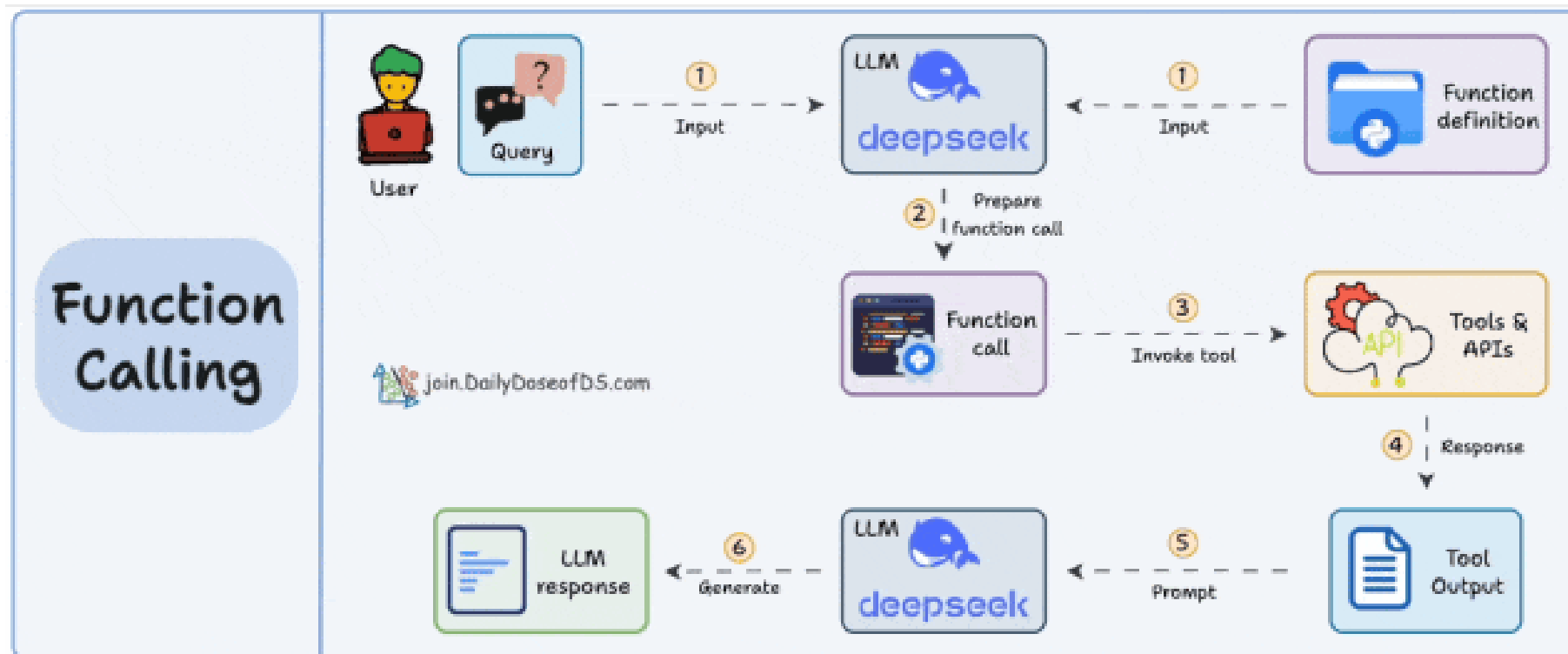




Function calling



Function Calling是通过自然语言让大模型智能地分析、调用外部函数或工具，以执行特定的操作。如去指定天气网站查询天气、文件读写、收发邮件等。这种功能可以解决信息实时性和数据局限性等问题。





Function calling 流程



❶ 定义大模型可用的工具函数

```
def  
get_current_temperature(...)  
    """ Args:  
        returns:  
    """  
    {  
        ...  
    }
```

❶ 通过tools字段告诉大模型“你可以使用的工具及工具的描述”

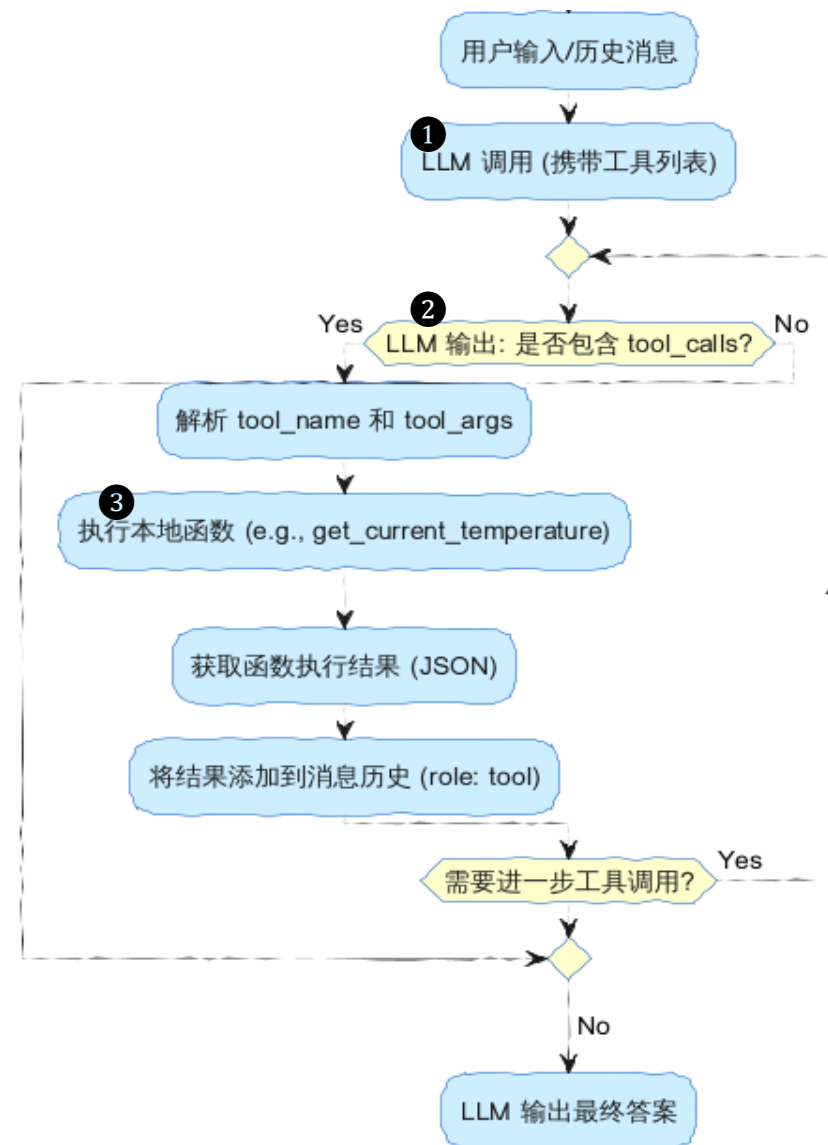
```
Response  
=client.chat.completions.  
create(model=MODEL_NAME,  
        messages=messages,  
        tools=tool_user)
```

❷ 大模型返回的 tool_calls 字段得到“大模型想要使用的工具及参数列表”

```
call_id = tool_call.id  
tool_name =  
tool_call.function.name
```

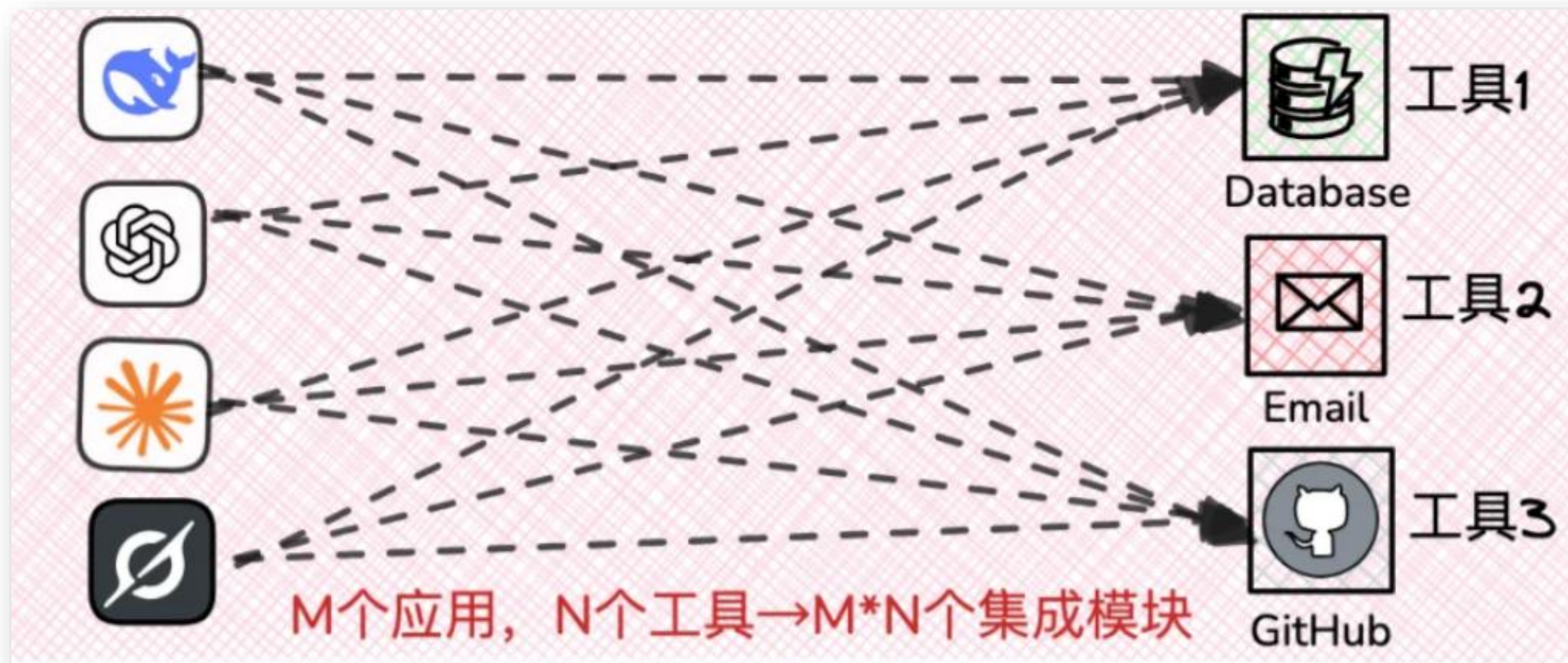
❸ 函数本地执行

```
fn_res = json.dumps  
(get_function_by_name(t  
ool_name)(**tool_args))
```





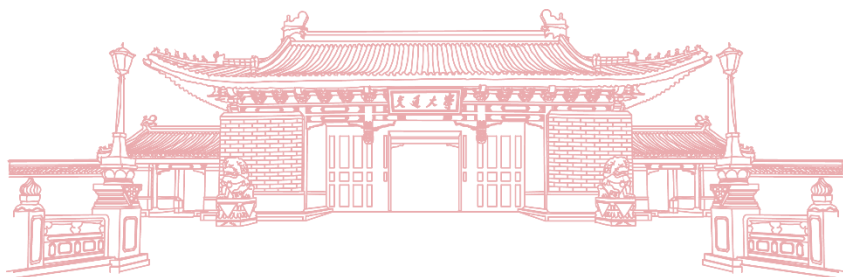
Function Calling 的缺点



Tools通常在会话开始时静态定义，会话中难以动态添加或移除工具。各功能需要单独开发，不通用。当业务不断增大，代码的维护性差、系统复杂性高。

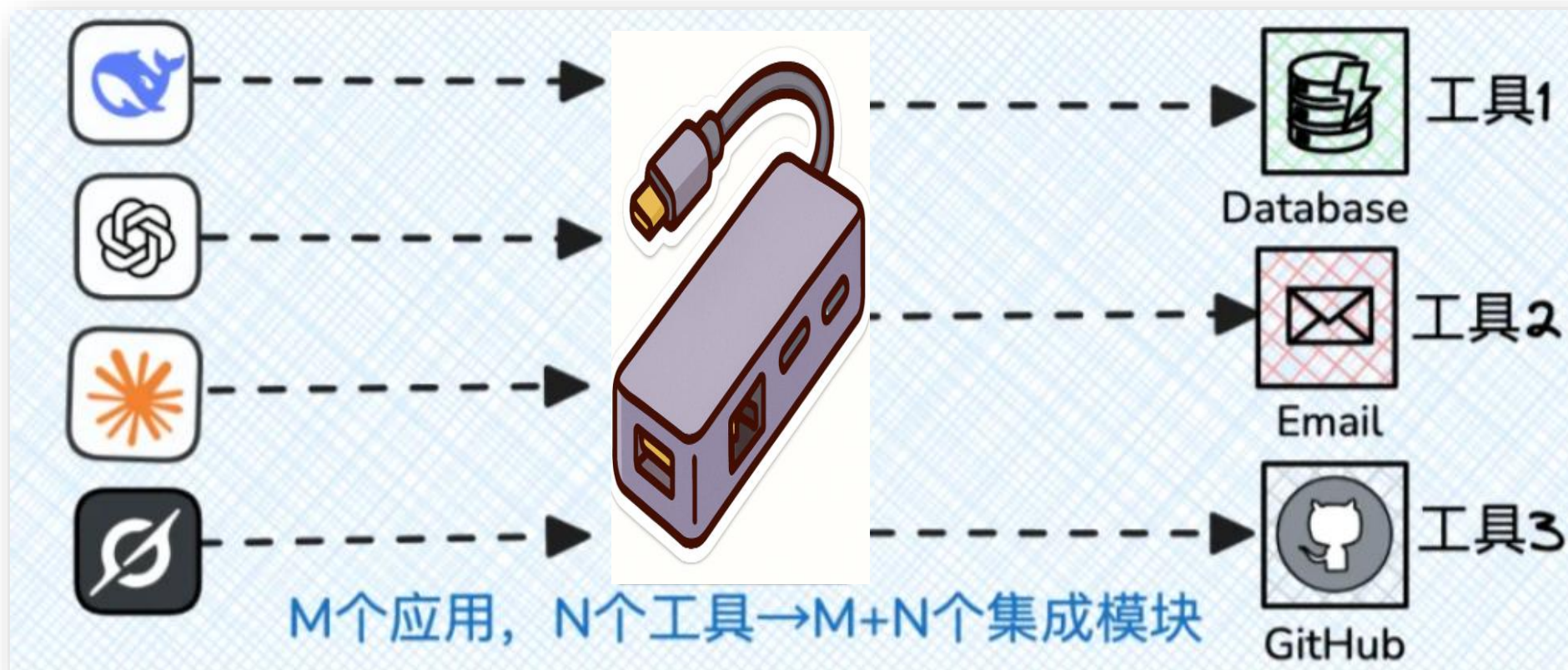


MCP（模型上下文协议）





MCP 协议的引入

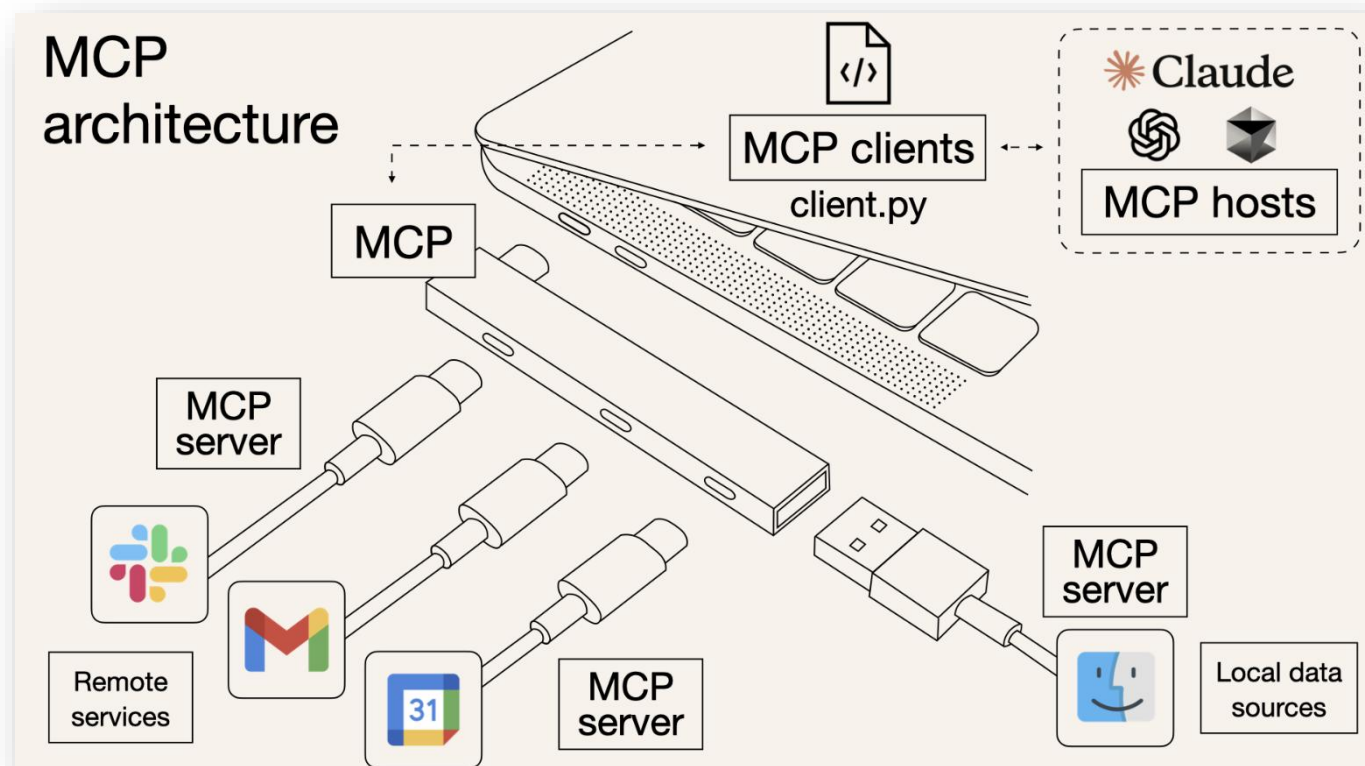


引入一个中间层，希望可以标准化接口设计，动态发现协议，客户端按需查询。为开发者提供了一个高效、灵活且可扩展的框架。类似于USB-C端口的通用连接特性。显著减少定制开发需求（例如将每个应用和每个工具的 $M \times N$ 次整合简化为 $M+N$ 次）。

MCP (模型上下文协议)



MCP (Model Context Protocol, 模型上下文协议) 起源于 2024 年 11 月 Anthropic 发布的文章: [Introducing the Model Context Protocol](#), 试图标准化function call的过程。它定义了应用程序和 AI 模型之间交换上下文信息的方式, 使得开发者能够以一致的方式将各种数据源、工具和功能连接到 AI 模型, 使 AI 应用程序的开发和集成变得更加简单和统一。

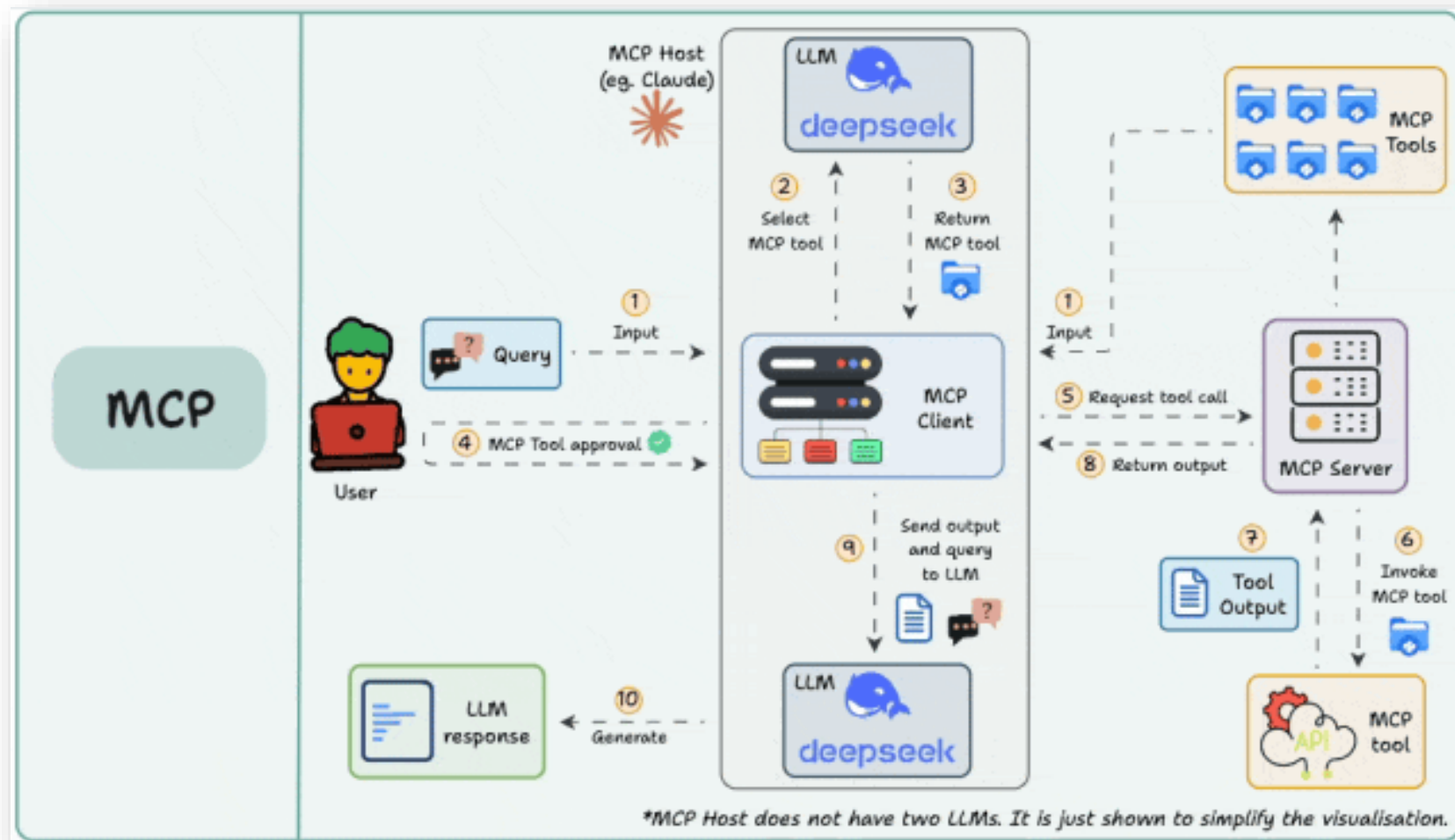


MCP 架构中的关键参与者包括：

MCP 宿主 (Host)：协调和管理一个或多个 MCP 客户端的 AI 应用，用于跟用户交互。如 Claude Code、Trae 等。

MCP 客户端 (Client)：维持与 MCP 服务端连接并获取上下文供 MCP 宿主使用的组件。

MCP 服务端 (Server)：为 MCP 客户端提供上下文的程序。可以在本地或远程执行，使用不同的传输协议。

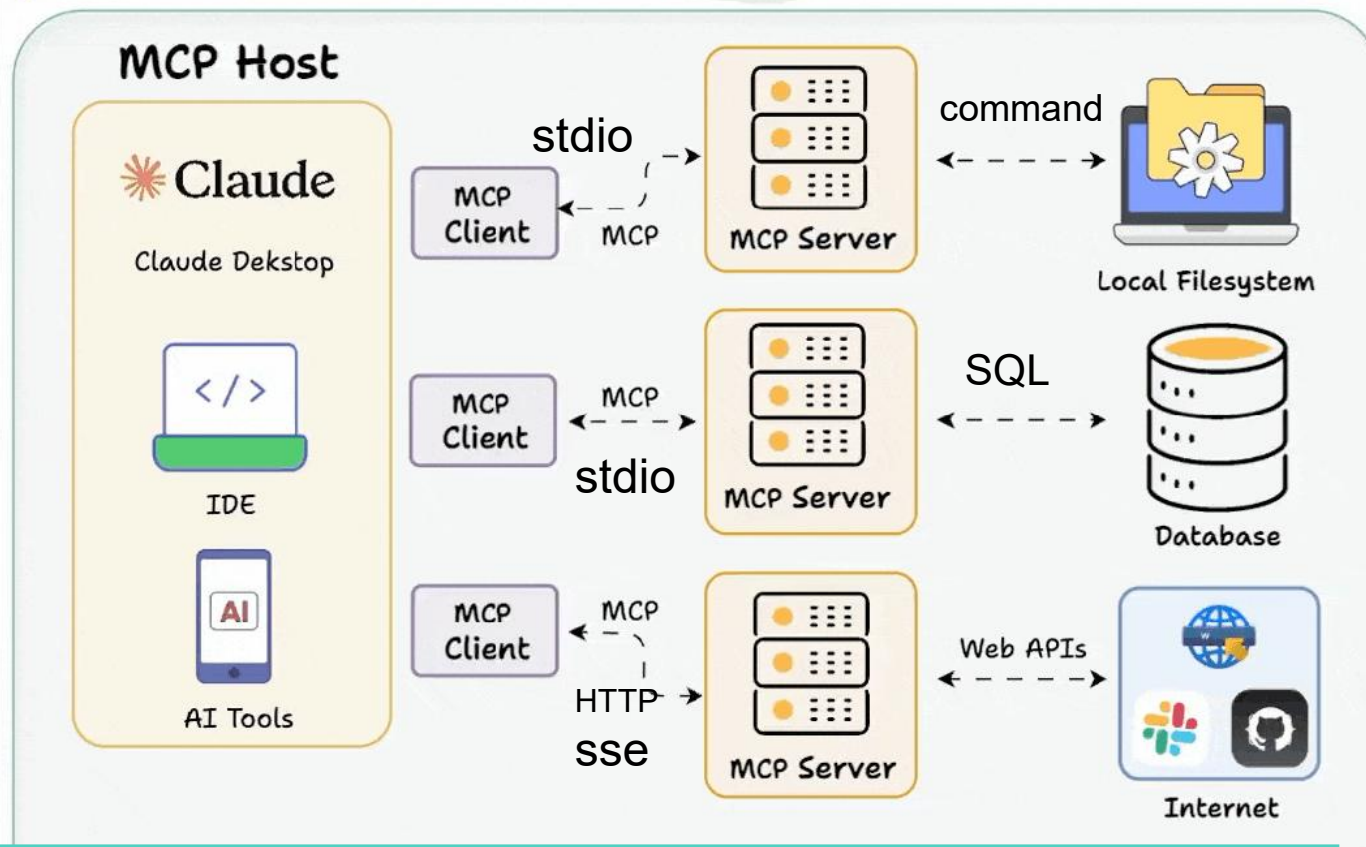


MCP 客户端由宿主应用程序实例化，用于与特定的 MCP 服务器通信。

MCP 服务端提供三个核心功能，工具、资源、提示。

MCP传输协议主要包括：SSE、STDIO。

Model Context Protocol (MCP), clearly explained



本地文件

数据库

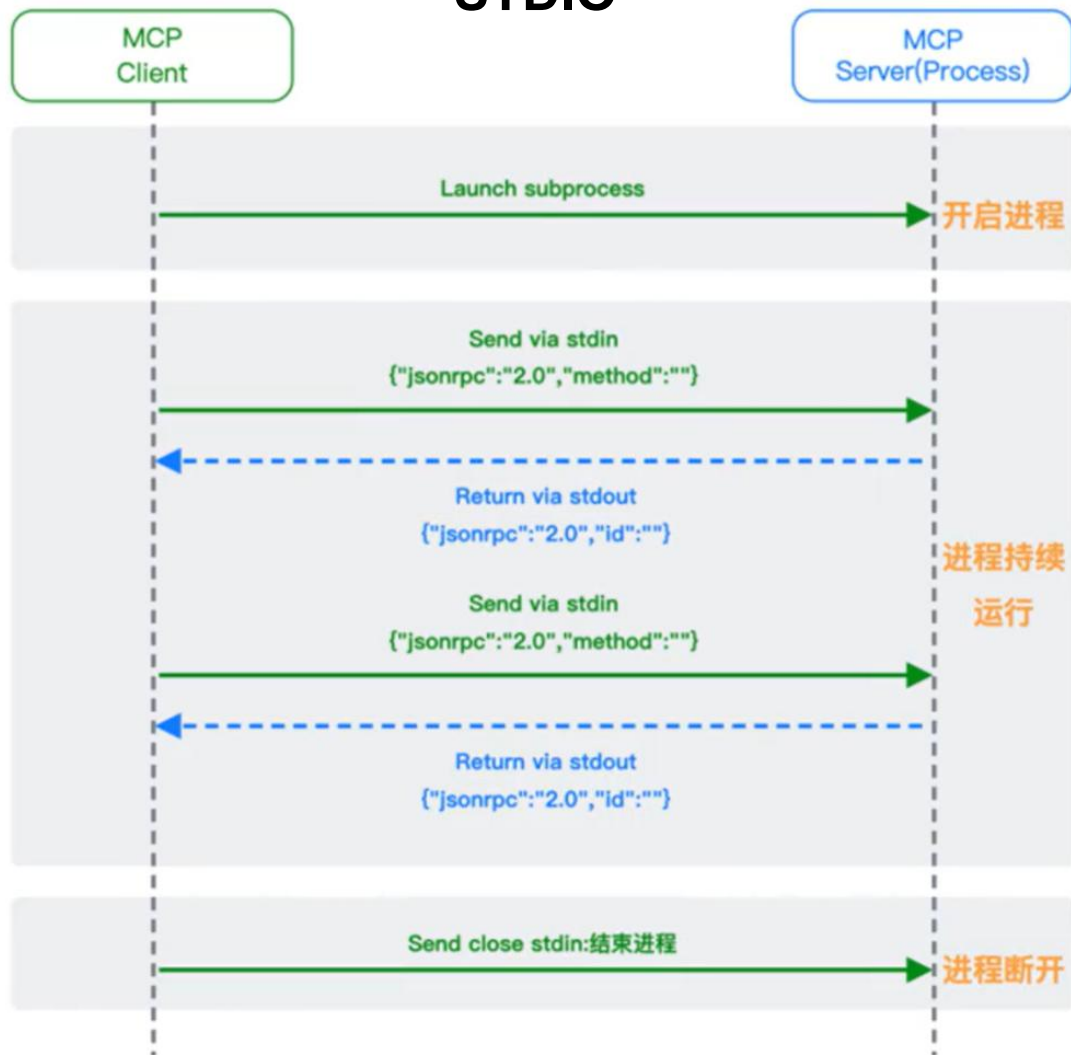
webAPI



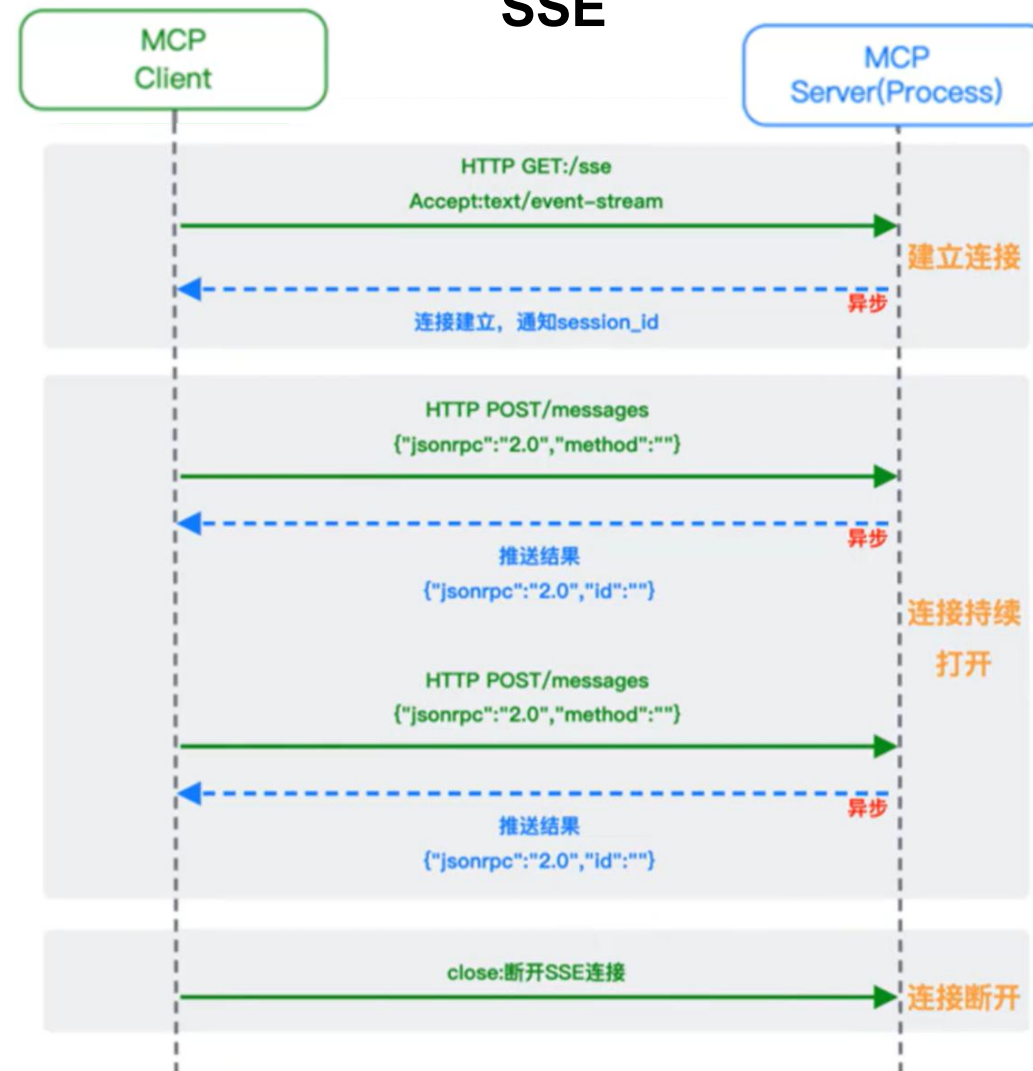
MCP传输模式



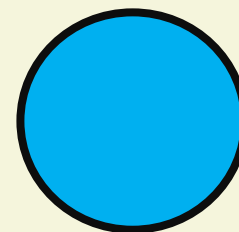
STDIO



SSE



动手实践时间



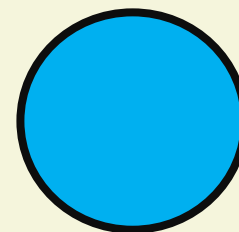
Demo Time!

Trae + 百度地图

根据提示安装node.js工具



动手实践时间



Demo Time!

2. Server + client 模式





MCP Server + client 启动



① 环境准备

`pip install uv` → Python包管理工具，快且易于应对复杂环境管理

`pip install fastmcp` → 无需手动定义JSON，用Python注解和docstring自动生成MCP工具

① 极速构建MCP Sever

```
import fastmcp as mcp
```

```
mcp = FastMCP("天气服务")
```

```
@mcp.tool()
```

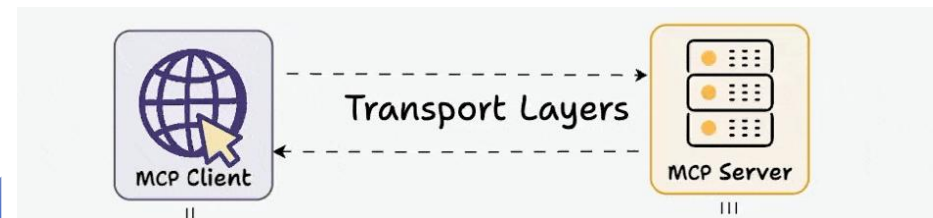
```
def get_weather(...):
```

```
    """获取天气"""
```

```
    return {"city": city, "temp": "25°C"}
```

```
mcp.run(transport="stdio")
```

装饰器:接受一个函数，返回一个新函数。
作用：在不修改原代码的情况下，给代码批量提取参数、类型、描述 并生成JSON Schema。形成大语言模型可以直接调用的工具。



Stdio模式: 本地传输，接收标准输入、输出
Sse模式: 流式传输技术，Sever部署在云端，边传输边处理。



MCP Server + client 启动



② 构建MCP Client

异步连接MCP Server: `load_servers`

`await` 等待异步操作结束

调用大模型进行聊天: `chat_loop`
(与function call 中的实现基本类似)

③ 配置.env文件

用Openai接口进行大模型调用,安装openai 包
`Uv add openai python-dotenv`

使用qwen-plus模型,
<https://dashscope.aliyun.com> ->控制台-
>API-KEY管理

填写base url和api_key

`server_config.json` 配置文件

`name:config` 键值对

```
"mcpServers": {  
  "research": {  
    "command": "uv",  
    "args": ["run", "research_server.py"]  
  }  
}
```

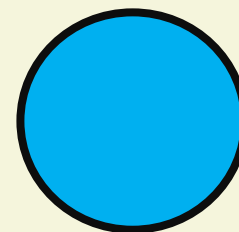
④ 启动MCP Client, 会自动拉起MCP Server, 通过CLI进行交互

`python mcp_chatbot.py`

或者

`uv run mcp_chatbot.py`

动手实践时间



Demo Time!

3. MCP进行论文查询、下载、邮件发送等功能

`pip install arxiv`





添加外部server，解锁无限能力



人工智能实践教学中心
Artificial Intelligence Practice Teaching Center

为什么要连接外部服务器？

能力扩展：

将LLM与外部工具（如数据库、API、文件系统）无缝集成。

功能复用：

利用社区或企业内部已部署的MCP服务，避免重复开发。

实时数据：

访问最新数据，确保响应的时效性和准确性。

优质MCP服务器社区：



官方：[modelcontextprotocol/servers: Model Context Protocol Servers](https://modelcontextprotocol.com/servers)



ModelScope：阿里巴巴达摩院AI模型社区



GitHub / Playwright：代码仓库操作、网页自动化。

如何添加现有MCP服务器？

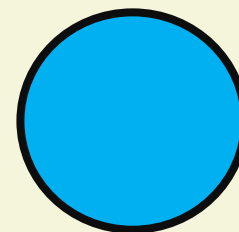
无需重写工具，只需修改配置文件

`server_config.json` 配置文件

增加 `server_name:config` 键值对

或者在host 应用中“添加更多工具”

动手实践时间



Demo Time!

4. 添加 12306/百度地图等mcp server





Context Window (上下文窗口)：语言模型输入（输出）的长度是有限的，每个语言模型的上限都不同，
（比较好的模型可以输入上百万token）
输入越长，就算没到上限，焦点模糊，无法准确完成任务理解。

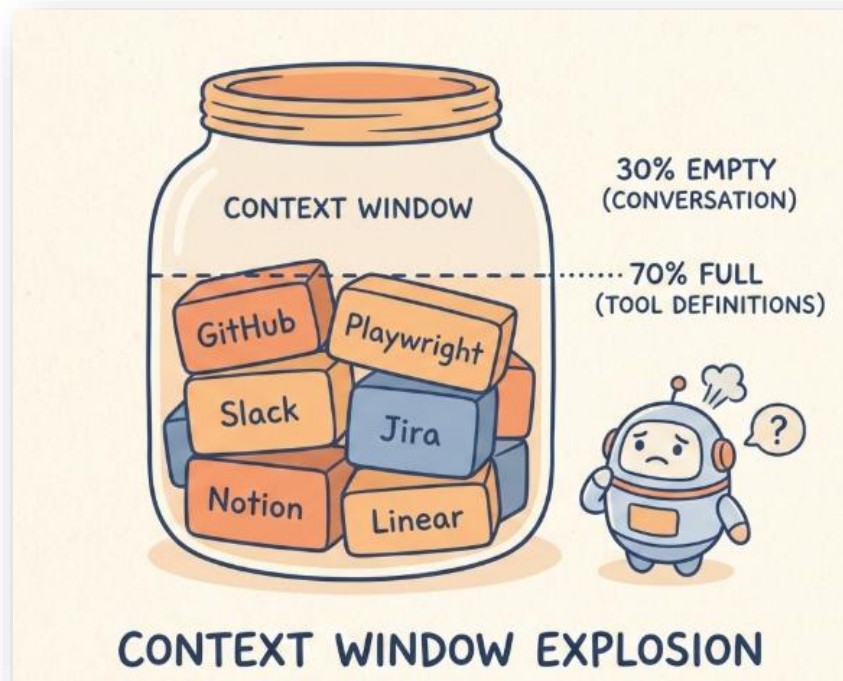
SYSTEM PROMPT + USER QUERY + HISTORY + TOOLS



上下文爆炸：每个 MCP Server 连接到 AI 时，必须把所有工具的定义（名称、描述、参数、示例）一次性塞进上下文。“把所有工具摆出来让 AI 挑”，一个工具的定义大概 500-800 tokens，一个 MCP Server 通常有 10-20 个工具。**工具使用准确性就会下降。**

每个工具都有详细的JSON Schema描述- 参数描述、类型定义、约束条件都很详细

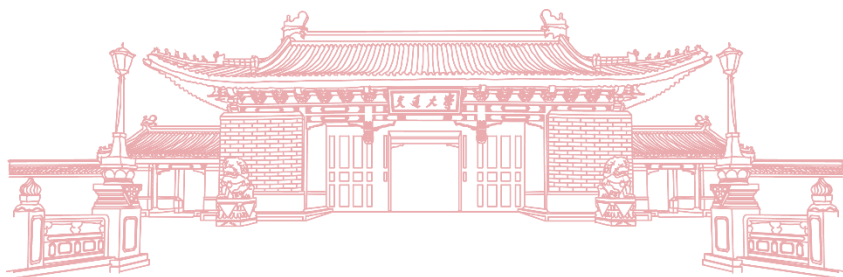
```
"name": "search_web",  
"description": "Search the web  
for information...",  
"parameters": {"type": "object",  
"properties": {"query": {"type":  
"string", "description": "Search  
query..."}, "num_results":...
```



- **GitHub MCP Server:**
27 个工具，消耗约
18,000 tokens
- **Playwright MCP
Server:** 21 个工具，
消耗约 13,600
tokens
- **mcp-omnisearch:**
20 个工具，消耗约
14,200 tokens

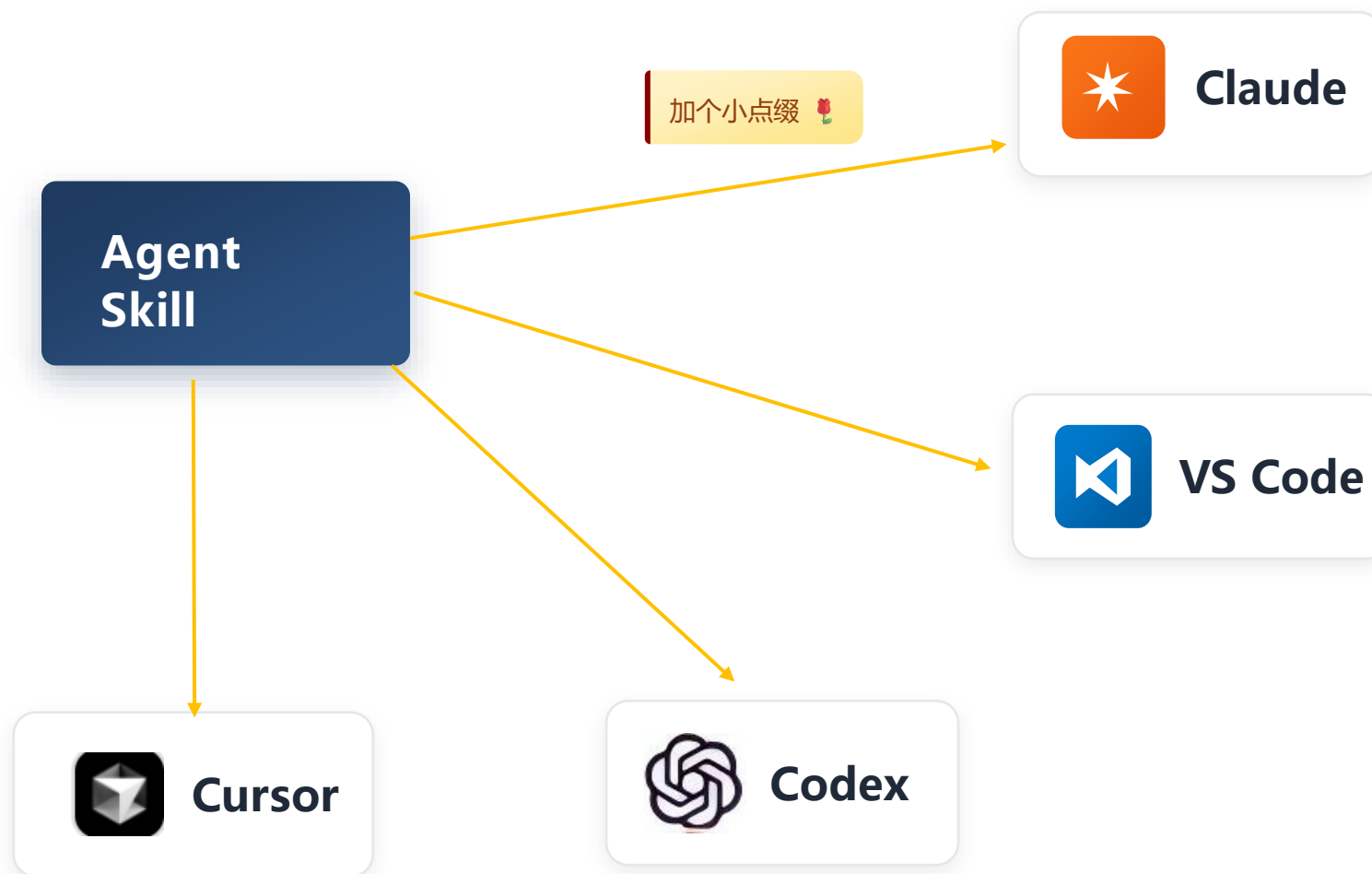


AGENT SKILLS (技能)





Agent Skill



Skill是 Anthropic 2025年12月推出的开放标准能力扩展包。



说明文档

回答时请注意：

1. 必须先安抚用户
2. 不得随意承诺

智能客服Skill

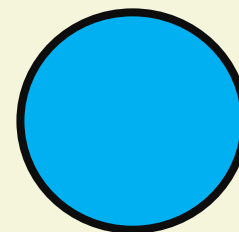
请讲会议内容总结
为如下几点：

1. 参会人员
2. 议题
3. 决定

会议总结skill

SKILLS 类似工作流程，它不是工具，但它可以使用工具。Skill 指令负责说明“如何在特定流程或多服务器工作流中使用这些工具”，让模型在特定场景下稳定、一致、可控地输出高质量结果。

动手实践时间



Demo Time!

Trae添加 会议内容记录skill 并测试

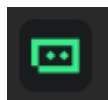




SKILL的运行流程



用户请求



Trae



大模型

"帮我总结以下
会议内容：....."

User:帮我总结以下会议内容：...
Agent Skills 列表（只有名称和描述）

选择“会议总结助手” Skill

User:帮我总结以下会议内容：...
“会议总结助手”的SKILL.md

总结内容如下：.....

总结内容如下：.....

SKILLS的特点是工具不再预加载，而是**按需发现**。

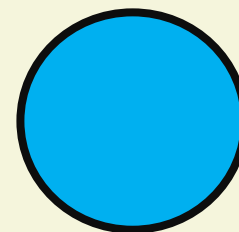


渐进式披露 (Progressive Disclosure) 机制：Skill 的元信息在上下文中，Agent 根据对话动态决策使用哪个技能，以及根据技能描述动态加载所需要的资源。



很多skills自带可执行脚本，对于大量本地任务，Skills + 内置工具就能完成，不需要额外的 MCP Server。

动手实践时间

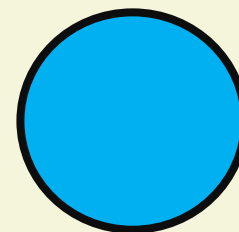


Demo Time!

Trae添加 会议内容记录skill 增加reference



动手实践时间



Demo Time!

Trae添加 会议内容记录skill 增加scripts





MCP connect Agent to data, Skills teach Agent what to do with that data.

Skills

Knowledge sharing - experience, best practices, workflows

- ✓ Simple markdown files - anyone can create
- ✓ Progressive loading - token efficient
- ✓ No server setup required
- ✓ Works across Web + Desktop + CLI (for most skills)

MCP

Functionality extension - connecting to APIs, databases, tools

- ✓ Requires coding and server setup
- ✓ Loads all tool definitions at startup
- ✓ Powerful for external integrations
- ✗ Higher token consumption, complex setup

SKILLS是经验、流程的封装，不仅教“用什么工具”，还教“怎么一步步做这件事”；优势在于**渐进式加载**，省Token。

MCP是连接与交互的协议，不定义任务逻辑。启动时加载所有工具定义，Tokens 消耗更高，使用起来更复杂。

两者互补，Agent 可以通过 Skills 获取知识，通过 MCP 拓展功能。



MCP

功能

给大模型提供数据

举例

查询昨天的销售记录

读取订单的物流状态

Agent Skill

功能

教大模型如何处理数据

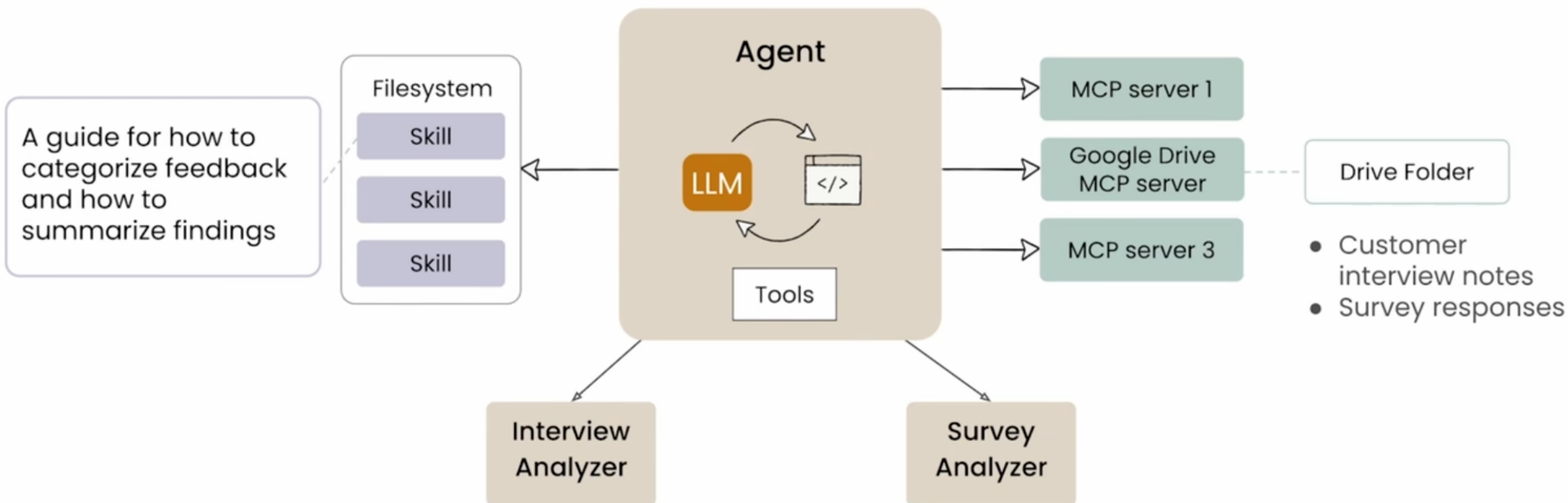
举例

会议总结的流程

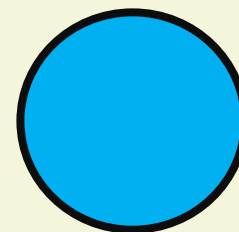
汇报文档撰写要求



How does Agent work



动手实践时间

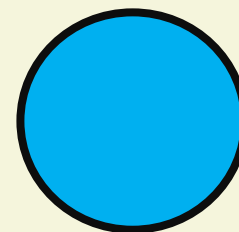


Demo Time!

使用skill+MCP 实现会议记录



动手实践时间

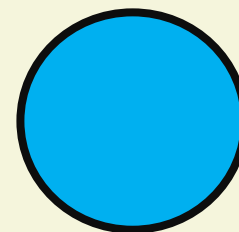


Demo Time!

使用public skills : PPT/xslx/pdf



动手实践时间



Demo Time!

使用skill creator 创建自己的skill





Claude官方推出的**创造技能**的技能

通过自然语言对话，AI 自动生成完整的代码、文档与测试套件

功能特点：

- 自动化技能创建流程
- 智能文件结构生成
- 代码与文档自动编写
- 实时测试与验证
- 迭代优化建议

工作流程：

- 提需求
- 智能文件结构生成
- 代码与文档自动编写
- 实时测试与验证
- 用户测试与反馈
- 迭代优化



1.提需求



对话示例：医疗记录 OCR 技能

 我需要一个医疗记录 OCR 技能

请描述具体需求，比如需要识别哪些类型的医疗文档？

 能识别病历、化验单、处方

需要支持哪些文件格式？

 PDF、JPG、PNG

是否需要结构化输出？

 是的，提取关键信息到表格



2. AI自动生成skills文件结构



```
● ● ● medical_record_ocr_skill/

📄 SKILL.md    # 技能定义与配置
📄 requirements.txt    # Python 依赖
📁 scripts/
  📄 ocr_medical_record.py    # OCR 核心模块
  📄 process_medical_text.py    # 医疗信息提取
  📄 generate_health_record.py    # 健康文档生成
  📄 set_reminder.py    # 提醒功能
📁 references/
  📄 medical_terms_correction.json    # 医疗术语修正
📄 sample_medical_record.txt    # 示例病历
```



3.分步创建并测试



1 核心功能开发

- </> **OCR 图像识别**
医疗文档图像识别
- </> **文本解析器**
医疗文本智能提取
- </> **数据结构化**
信息格式化输出

2 单元测试编写

- 🔧 **识别准确率**
图像识别精度测试
- 🔧 **解析正确性**
文本提取验证
- 🔧 **边界条件**
异常处理测试

3 集成测试

- 🔗 **端到端流程**
完整链路测试
- 🔗 **性能压力**
高负载测试
- 🔗 **错误处理**
异常恢复测试

Todos

- [✓] 分析病历扫描OCR技能需求
- [✓] 创建技能目录结构
- [•] 编写 **SKILL.md** 核心技能文档
- [] 创建测试用例和验证脚本
- [] 测试技能功能



4. Skill 交付



技能结构

```
medical_record_ocr_skill/
├── SKILL.md                # 技能核心文档（详细工作流程）
├── INSTALL.md              # 安装指南（依赖安装步骤）
├── USAGE_EXAMPLES.md       # 使用示例（多种场景）
├── README.md               # 项目概述
├── scripts/                # 核心脚本
│   ├── ocr_medical_record.py # OCR识别与修正
│   ├── process_medical_text.py # 医疗信息提取
│   ├── generate_health_record.py # 健康文档生成
│   ├── set_reminder.py       # 复诊提醒系统
│   └── test_workflow.py      # 完整测试套件
├── references/              # 参考文件
│   └── medical_terms_correction.json # 医疗术语修正词典
└── examples/                # 示例文件
    └── sample_medical_record.txt # 示例病历
```

核心特性

- **智能纠错**：内置200+医学术语修正规则
- **多格式支持**：图片、PDF、扫描件
- **隐私保护**：本地处理，不上传云端
- **批量处理**：支持多文件批量操作
- **可扩展性**：可自定义医疗术语词典



交付清单



完整源代码包



详细技术文档



测试用例套件



使用示例文件



配置文件模板



部署指南



Skill Creator 建立了完整的迭代优化闭环，AI 持续收集用户反馈，分析问题，生成优化方案，确保技能不断进化。



问题解决方案总结

您遇到的错误 `tesseract is not installed or it's not in your PATH` 是因为缺少OCR引擎。以下是完整的解决方案：

方案一：安装Tesseract OCR（完整功能）

Windows用户：
1. 下载安装程序：<https://github.com/UB-Mannheim/tesseract/wiki>
2. 运行安装：双击安装，使用默认路径 `C:\Program Files\Tesseract-OCR`
3. 添加到PATH：
临时添加（当前会话）
`set PATH=%PATH%;C:\Program Files\Tesseract-OCR`
或永久添加：
1. 右键“此电脑”→“属性”→“高级系统设置”
2. 点击“环境变量”→“编辑“Path”
3. 添加：`C:\Program Files\Tesseract-OCR`

你这个还是不行 我是中文的病例 但是你处理出来全是一堆英文字母 没有任何意义 昨天你创建的skill 用easyocr 还是能处理中文的 这版怎么不行了！

你说得对！我发现了问题。改进版脚本使用了EasyOCR，但可能配置有问题。让我检查并修复这个问题。

Todos

- [•] 检查改进版OCR脚本的中文识别问题
- [] 修复EasyOCR中文识别配置
- [] 测试中文病历识别效果



Skill creator- 最终完成任务



人工智能实践教学中心
Artificial Intelligence Practice Teaching Center



用户指令

简单一句话

"帮我扫描 test.jpg 并记录提醒我"



Skill 自动执行

多步骤自动完成

- 1 OCR 识别医疗文档
- 2 提取关键医疗信息
- 3 结构化保存记录
- 4 生成提醒事项

健康管理建议

短期 (1个月内)

1. ☒ 完成当前治疗方案
2. ☒ 注意足部卫生, 保持干燥
3. ☒ 避免穿紧身或不透气的鞋子
4. ☒ 定期观察足部变化

中期 (1-3个月)

1. 按医生建议时间复诊
2. 记录治疗反应和症状变化
3. 如有异常及时就医

长期 (3个月以上)

1. 建立完整的健康档案
2. 定期回顾健康数据
3. 制定个性化健康计划

紧急情况处理

需要立即就医的情况:

- 足部出现剧烈疼痛
- 感染迹象 (红肿、发热、流脓)
- 症状突然加重
- 发烧等全身症状



Open claw use case 社区



[awesome-openclaw-usecases](#)

[awesome-openclaw-usecases-zh](#)



谢谢！